MongoDB × Viam Robotics Hackathon

Builder Resources & Field Guide

Build one useful end-to-end slice. Pick real robot or sensor data, store and shape it in MongoDB, and show a useful result: a dashboard, an alert, a status summary, or a searchable history. You don't need every tool on this page. Treat it as a menu and pick what fits your idea.

Why MongoDB for robotics data

MongoDB Atlas combines a flexible document model, a powerful aggregation framework, and built-in search and vector search in one platform. Robot data is a natural fit: readings, events, and state vary by device and change over time, and documents let you store them as they arrive without forcing a rigid schema. As projects move from simple retrieval to AI agents, Atlas can also act as both long-term and short-term memory for those agents.

1. Get oriented

Use these to get a database running and understand how MongoDB organizes data before you build.

- Sample Mflix dataset: Ready-made data you can load into Atlas to practice queries and prototype quickly when your own data isn't flowing yet. sample_mflix.embedded_movies already includes vector embeddings, so it works for Vector Search practice too. It is movie data, not hackathon data.
- MongoDB Tools: Command-line utilities for importing, exporting, and moving data in and out of MongoDB. Handy for loading a CSV or JSON export of your readings.
- Getting Started with MongoDB Atlas and Python: A hands-on notebook for connecting a Python app to Atlas. A good first stop if your robot code or analysis is in Python.
- AI Learning Hub: Guides and tutorials for building AI applications with MongoDB.

2. Shape and analyze your data

Decide how your readings and events are stored, then summarize them. This is where most hackathon projects get their value.

- Data Modeling in MongoDB: Best practices for structuring documents. Use it when deciding how to record robot runs, observations, actions, timestamps, and outcomes so they are easy to query later.
- MongoDB Aggregations: Pipelines that filter, group, and calculate across many documents. Use them to turn raw readings into counts, averages, trends, or alerts for your app.

3. Search and find similar

Add these when your app needs to look things up by meaning or by text, not just by exact field values.

- MongoDB Atlas Search: Full-text search inside your database. Useful for searching logs, notes, or event descriptions with typo tolerance and relevance ranking.
- MongoDB Vector Search: Finds items that are similar in meaning rather than identical. Useful for questions like "find observations like this one" or searching maintenance notes in plain language.

- Perform Semantic Search on Your Data Using MongoDB Atlas: A tutorial that walks through semantic search end to end, from embeddings to queries.
- Embeddings Generator: A quick way to create the embeddings Vector Search needs.

4. Sample apps and code to build on

Start from a working example instead of a blank page. Pick one that matches the stack your team already knows.

- MongoDB GenAI Showcase: Examples, sample code, and cookbooks for integrating Atlas with frameworks like LangChain and LlamaIndex and with model providers such as OpenAI, Cohere, and Hugging Face.
- GenAI Showcase Chatbot: A chatbot that helps you find the right example in the Showcase.
- MERN stack example and MEAN stack example: Full-stack CRUD apps using MongoDB, Express, Node.js, and React or Angular. A fast starting point for a dashboard that displays robot data.
- MongoDB Chatbot Framework: Libraries for building full-stack chatbot applications on Atlas Vector Search, if your idea includes a conversational interface.

5. Optional stretch: RAG, memory, and agents

Retrieval-augmented generation (RAG) lets an AI model answer using your own data. Memory lets an agent remember past interactions. Explore these only if they solve a real problem in your project, for example a robot assistant that answers questions about its own history.

- How to Choose the Right Embedding Model for RAG: Guidance on picking an embedding model.
- Building a RAG System using LlamaIndex, OpenAI, and MongoDB Atlas: A step-by-step RAG build.
- How to Evaluate Your RAG Application: How to test and tune retrieval quality.
- MongoDB-RAG NPM package: A Node.js module for vector search, caching, batch processing, and indexing with Atlas.
- Semantic caching and memory with Atlas and LangChain: Speeds up repeated questions and keeps conversation history.
- Add memory to a JavaScript RAG application: The same idea for JavaScript apps.
- Building an Interactive RAG Agent with MongoDB Atlas and Function Calling: Shows an agent that decides when to query your data. Relevant if your robot or app should take actions based on what it finds.
- Building a PDF chatbot with Mistral AI and MongoDB Atlas: A compact chatbot example you can adapt to other documents, such as robot manuals.